(continued)

If you are using Google AI, there is a separate file named agents.google.js in the same folder:

```
$ node agents.google.js
```

You'll need an OpenAI API key to access their REST API or a Google API key for Google services. For details on setting up and creating API keys, see the appendix.

```
// .env

OPENAI_API_KEY=<SECRET_KEY>
GEMINI_API_KEY=<YOUR_GEMINI_API_KEY>
```

In the provided code, we create an instance of a retriever, which is then transformed into a tool using the `createRetrieverTool` function from LangChain. By doing this, we can seamlessly integrate the existing retrieval functionality into our agent creation process.

The `createRetrieverTool` function allows us to define a specific name and description for the tool, enhancing its usability and clarity when interacting with the AI model. The agent will try to use this tool description as a hint about whether it needs to call this tool and based on the predefined system prompt message that it follows.

The rest of the code is like the one shown in figure 6.7. I've also included code that shows how to stream the results back to the console.

Two things can affect the effectiveness and usability of these agents. First are *context length limitations*. The number of tools you can incorporate into a LangChain agent is constrained by the context length of the language model being used. Each tool's name, description, and arguments contribute to the total token count in the prompt, so if you have many tools with different contexts, you might see problems with maximum context window limits.

Second, there are *delays* and *propagation effects*. *Propagation* refers to the way information flows through the various components of the agent. In a typical LangChain setup, data must be passed through multiple layers—such as retrievers, tools, and language models—before reaching the final output. Each step in this chain can introduce additional latency and make our application respond slower. There are several ways to improve the efficiency of the LLM responses, including minimizing unnecessary computations and using caching mechanisms where appropriate. In chapter 8, we will explore best practices and advanced techniques to address these challenges.

6.4.3 Agent integration with the Vercel AI SDK

Once we have an instance of an agent configured, we can integrate this with the Vercel AI SDK in a manner similar to the way we streamed the chaining calls for the weather tool earlier in this chapter.